



CITS5017 Deep Learning Semester 2, 2026

Lab Sheet 4 (Not assessed)

1. Outline

This labsheet is a good exercise to practise what you learned from the lecture notes for Topic 4 (Chapter 15 “*Processing Sequences Using RNNs and CNNs*” of the textbook).

2. Tasks

Task 1

The first task for this week is to go through the Jupyter Notebook file for Chapter 15 supplied by Aurélien Géron on the GitHub page <https://github.com/ageron/handson-ml3>. This task will help complete tasks 2 and 3 later on.

Task 2

A numpy data file named *noisy_squares.npy* can be found on the LMS page of the unit. Your task is to train a simple RNN for binary classification. Download the data file and create a new Jupyter Notebook file for the labsheet. Copy the following function to a code cell:

```
def load_data(filename):  
    with open(filename, "rb") as f:  
        xtrain = np.load(f)  
        ytrain = np.load(f)  
        xval = np.load(f)  
        yval = np.load(f)  
    return xtrain, ytrain, xval, yval
```

Call the function to create your training and validation sets as follows (assuming that you have saved *noisy_squares.npy* in the same directory where you started *jupyter-lab*):

```
X_train, y_train, X_val, y_val = load_data("noisy_squares.npy")
```

You should find that the shapes of `X_train`, `X_val`, `y_train`, and `y_val` are, respectively, $(200, 4, 2)$, $(60, 4, 2)$, $(200,)$, and $(60,)$. The dataset contains 200 training instances, each of which is a sequence of 2D coordinates of four noisy vertices (i.e., the vertex coordinates have been perturbed by noise) of a square. Any of the four vertices of a square can be the starting vertex and the sequence can be in the clockwise or anti-clockwise order. To make the task easy for you, the training set has been synthesized to be perfectly balanced: there are 100 clockwise and 100 anti-clockwise sequences. Similarly, there are 30 clockwise and 30 anti-clockwise sequences in the validation set. In the `y_train` and `y_val` arrays, clockwise sequences are labelled as 1 and anti-clockwise sequences are labelled as 0.

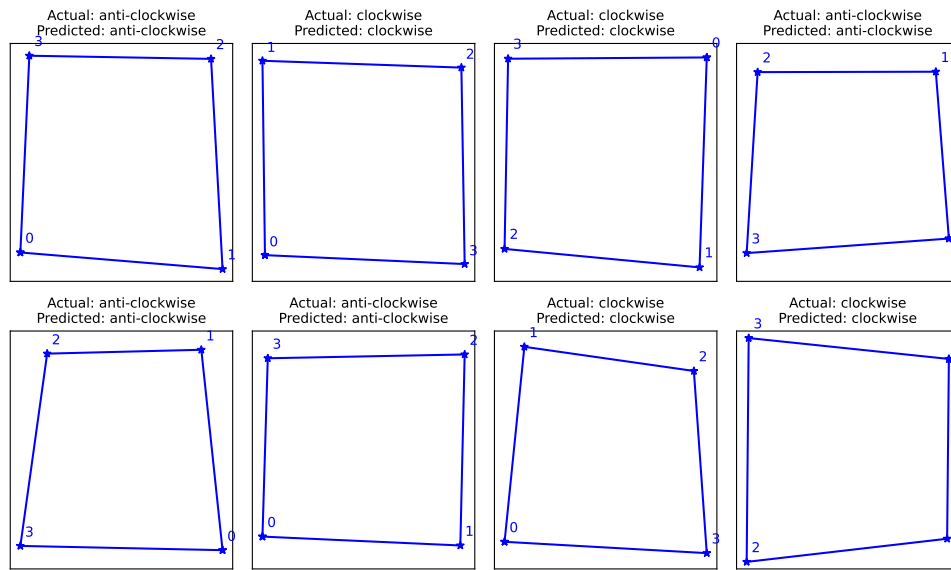


Figure 1: Example predicted results from the model in task 2.

Your task is to implement a simple RNN (e.g., having just 16 neurons should be sufficient) which should take the sequences of 4 vertices (i.e., 4 time steps) from the training set as input and predict whether each sequence has the vertices in the clockwise or anti-clockwise order.

Try training your model for 50 epochs with early stopping¹ and (e.g., by monitoring the validation loss). This is a very simple and fun dataset. You should be able to achieve 100% validation accuracy easily.

Hint: use the momentum optimizer and set the learning rate to around 0.01.

Show the learning curves at the end of training. Write a small function to display a few predicted results randomly sampled from the validation set. For visualisation purpose, you could join the last vertex back to first vertex so that you have a closed square for each case, like the diagram shown in Figure 1.

Task 3

Again, use the *noisy_squares* dataset above except that a different RNN should be trained to predict the 4th vertex coordinates using the first three vertex coordinates as input (i.e., over 3 time steps).

For this task, the input should be `X_train[:, :3, :]` (whose shape is (200, 3, 2)) and the output should be `X_train[:, 3, :]` (whose shape is (200, 2)). The variables `y_train` and `y_val` are not needed.

Again, a simple RNN with around 16 neurons should be sufficient. You should train your model for 50 epochs with early stopping (see footnote 1), monitoring the validation loss. Again, you can use the momentum optimizer with the learning rate set to 0.01. Show the learning curves.

Similarly to task 2 above, randomly sample a few instances from the predicted outputs for the validation set. Write a small function to display the results, like the diagram shown in Figure 2.

¹As this is a very easy and simple dataset, you might find that the training loss keeps coming down even at the last epoch while the accuracy is already at 100%. This means that the early stopping callback is probably not required.

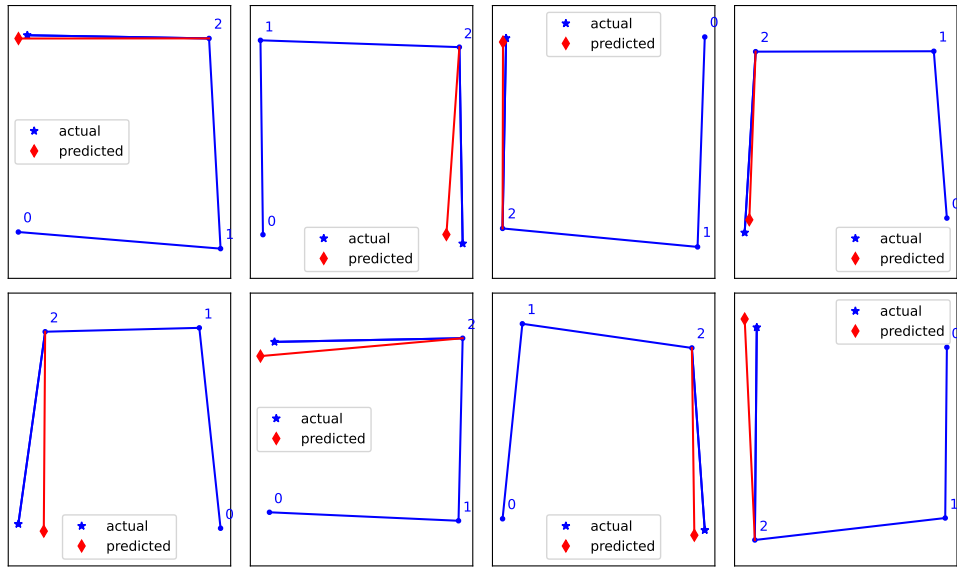


Figure 2: Example predicted results from the model in task 3.